Criterio	Excelente (10)	Bueno (8)
Arquitectura y Diseño (Modelo 4+1 Vistas)	Las 4 vistas (Lógica, Procesos, Implementación, Despliegue) están completamente documentadas con diagramas claros y coherentes. Los casos de uso unifican todas las perspectivas.	3 de las 4 vistas están bien documentadas; falta detalle menor en una de ellas. Los casos de uso están presentes pero con coherencia parcial.
Claridad Arquitectónica	Diagrama de capas (presentación, servicios, lógica de negocio, datos, persistencia) es detallado, preciso y bien anotado. Las responsabilidades están claramente delimitadas.	Diagrama de capas presente con buena separación de responsabilidades. Faltan anotaciones menores o detalle en algunas capas.
Componentes Reutilizables	Se identifican y documentan ≥ 7 componentes. Cada uno cuenta con interfaz documentada y se demuestra su uso en ≥ 3 contextos.	Se implementan 5-6 componentes reutilizables. Documentación clara. Se demuestran en 2+ contextos.
Cobertura Funcional de Módulos	Todos los 7 módulos funcionales con $\geq 90\%$ de funcionalidades especificadas.	6 de 7 módulos completos; 1 módulo tiene cobertura 70-80%. Funcionalidades críticas implementadas.
Integración de Procesos de Negocio	Los 5 procesos clave están completamente integrados. Los flujos end-to-end funcionan sin interrupciones.	4 de 5 procesos integrados con flujos completos. 1 proceso tiene integración parcial.
Integridad de Datos	Base de datos normalizada (3FN mínimo). Claves foráneas, restricciones y validaciones funcionan correctamente. Auditoría registra todos los cambios. Transacciones garantizadas.	Base de datos bien normalizada. Algunas restricciones faltan o hay inconsistencias menores. Auditoría implementada.

Manejo de Errores y Validaciones	Validaciones robustas en frontend y backend. Mensajes de error claros y útiles. Casos excepcionales cubiertos. Recuperación de errores implementada.	Validaciones presentes en ambas capas. Mensajes útiles. Casos excepcionales parcialmente cubiertos.
Calidad de Código	Código limpio. Patrones de diseño aplicados (MVC, DAO, Factory). Componentes modularizados. Fácil de leer y mantener.	Código generalmente limpio. Patrones aplicados en la mayoría. Modularización adecuada. Algunos comentarios útiles.
Uso de patrones de diseño	Implementa consistentemente funciones puras, inmutabilidad, composición y reactividad. Código altamente modular y sin efectos secundarios.	Aplica correctamente funciones puras, composición e inmutabilidad. Mínimos efectos secundarios. Domina conceptos fundamentales.
Uso de programación funcional	Aplica múltiples patrones apropiados (Singleton, Factory, Observer, Strategy). Selección justificada. Mejora significativamente la arquitectura.	Implementa varios patrones relevantes y correctos. Mejoran la estructura sin complejidad innecesaria.
Documentación Arquitectónica (SAD)	Documento completo con: descripción de vistas 4+1, decisiones arquitectónicas justificadas, diagramas UML (casos de uso, clases, secuencia, componentes, despliegue), matriz de trazabilidad.	Documento bien estructurado con 4 de las 5 vistas detalladas. Diagramas UML principales presentes. Justificaciones claras.
Especificación de Requisitos (SRS)	Documento detallado con requisitos funcionales y no funcionales claramente especificados. Criterios de aceptación explícitos. Matriz de trazabilidad requisitos-pruebas-código.	Documento claro con requisitos funcionales y no funcionales. Criterios de aceptación presentes. Matriz básica incluida.

Manual Técnico	Guía completa: instalación paso a paso, configuración de variables de entorno, despliegue en desarrollo y producción, documentación API Swagger/OpenAPI, guía de mantención y extensión.	Guía con pasos claros para instalación y despliegue. API documentada. Algunas secciones avanzadas incompletas.
Manual de Usuario	Manuales paso a paso por cada módulo. Capturas anotadas de cada funcionalidad. Procedimientos de casos de uso críticos. Glosario de términos de negocio. Guía de resolución de problemas.	Manuales para módulos principales. Capturas presentes. Procedimientos claros. Glosario básico.
Pruebas Unitarias y Cobertura	Cobertura ≥ 80% de código. Pruebas bien estructuradas con casos positivos y negativos. Reportes documentados. Integración continua configurada.	Cobertura 70-79%. Pruebas unitarias implementadas. Reportes básicos generados.
Control de Versiones (Git)	Historial claro con commits descriptivos. Ramas bien organizadas. Merge coherente. Rama main estable con código entregable. README instructivo.	Repositorio bien organizado. Commits claros. Rama main estable. Documentación básica en README.
Plan y Ejecución de Pruebas	Plan de pruebas detallado con casos documentados. Reportes de ejecución con resultados, defectos encontrados y soluciones. Pruebas de carga/rendimiento. Matriz de trazabilidad requisitos-pruebas.	Plan de pruebas claro con casos documentados. Reportes de ejecución. Algunas pruebas de rendimiento.
Presentación Final (Demo + Justificación)	Demostración fluida de todos los módulos. Sistema operativo sin errores visibles. Justificación clara de decisiones arquitectónicas. Lecciones aprendidas y mejoras realistas. Todos los integrantes participan activamente.	Demo completa de módulos clave. Sistema operativo. Justificación presente. Lecciones documentadas. Mayoría del equipo participa.
Participación Equitativa del Equipo	Todos los integrantes tienen contribuciones claras y documentadas en código (commits visibles), documentación y presentación. Cada persona explica su parte.	Todos participan activamente. La mayoría tiene contribuciones visibles. Puede explicar su trabajo.

Satisfactorio (6)	Deficiente (4)	Insuficiente (0-2)	Ponderación	Puntuación Asignada
2 de las 4 vistas están documentadas adecuadamente. La documentación es básica pero comprensible.	Solo 1 vista está documentada o las documentadas carecen de claridad. Falta integración de casos de uso.	Faltan 2 o más vistas. La documentación es confusa o inexistente.	30%	
Diagrama básico de capas presente. La separación de responsabilidades es evidente pero podría ser más clara.	Diagrama vago o incompleto. La separación de responsabilidades es confusa.	No hay diagrama arquitectónico o es incomprensible.	Incluido en 30%	
Se identifican e implementan 4-5 componentes. Documentación básica. Reutilización demostrada en ≥ 2 módulos.	2-3 componentes identificados pero con implementación incompleta o documentación pobre. Reutilización limitada.	Menos de 2 componentes o no hay evidencia de reutilización.	20%	
5 de 7 módulos con cobertura 80%+. Módulos restantes tienen funcionalidad básica (≥60%).	3-4 módulos funcionales; cobertura 50-70% en la mayoría. Faltan funcionalidades críticas.	< 3 módulos o cobertura < 50%. Sistema no operativo.	Incluido en 30%	
3 de 5 procesos con integración completa. Los 2 restantes tienen integración básica.	2-3 procesos con integración parcial. Múltiples desconexiones.	< 2 procesos integrados o integración inexistente.	Incluido en 30%	
Base de datos normalizada básicamente. Algunas validaciones ausentes. Auditoría parcial.	Base de datos con denormalizaciones o validaciones débiles. Auditoría incompleta.	Base de datos desorganizada o sin validaciones.	Incluido en 30%	

Validaciones básicas presentes. Mensajes genéricos. Algunos casos excepcionales omitidos.	Validaciones débiles o solo en una capa. Mensajes poco útiles.	Validaciones ausentes o no funcionales.	Incluido en 30%
Código legible. Patrones parcialmente aplicados. Modularización básica. Comentarios escasos.	Código desorganizado. Patrones poco evidentes. Modularización débil. Difícil de mantener.	Código ilegible o desorganizado. Sin aplicación de patrones.	10%
Usa algunas características funcionales (map, filter, reduce) pero de forma inconsistente. Mezcla paradigmas sin criterio claro.	Intenta conceptos funcionales con errores. Código predominantemente imperativo. Confusión entre paradigmas.	No usa programación funcional o la implementa incorrectamente.	5%
Usa uno o dos patrones básicos y correctos. Aplicación funcional pero podría optimizarse.	Intenta patrones con errores de implementación o selección inadecuada. No mejora el diseño.	No usa patrones o los implementa incorrectamente.	5%
Documento presente con 3 vistas documentadas. Diagramas básicos. Justificaciones parciales.	Documento incompleto. 2 vistas o menos documentadas. Diagramas faltantes o vagos.	Sin documentación arquitectónica o muy superficial.	10%
Documento con requisitos listados. Criterios de aceptación parciales. Trazabilidad simplificada.	Documento vago o incompleto. Muchos requisitos sin criterios claros.	Sin SRS o muy deficiente.	10%

Guía básica presente. Instalación clara. API documentada parcialmente. Mantenimiento no documentado.	Guía incompleta o confusa. Pasos faltantes. Documentación API débil.	Sin manual técnico o inservible.	5%
Manuales presentes pero con detalle limitado. Capturas incompletas. Procedimientos simplificados.	Documentación de usuario mínima o confusa. Capturas faltantes.	Sin manual de usuario.	5%
Cobertura 60-69%. Pruebas unitarias presentes pero parciales. Reportes simples.	Cobertura 40-59%. Pruebas limitadas o mal diseñadas.	Cobertura < 40% o sin pruebas unitarias.	10% (Opcional)
Repositorio funcional. Commits presentes pero descripciones vagas. Rama main sin issues críticos.	Repositorio desorganizado. Commits frecuentes sin descripción. Rama main inestable.	No hay versioning o está caótico.	10% (Opcional)
Plan básico de pruebas. Casos documentados parcialmente. Reportes simples.	Plan de pruebas vago. Pocos casos documentados. Reportes faltantes.	Sin plan de pruebas formales.	10% (Opcional)
Demo de módulos principales. Sistema mayormente operativo. Justificación básica. Lecciones simples.	Demo incompleta o con errores frecuentes. Justificación débil. Participación parcial.	Demo fallida o no realizada. Justificación ausente.	10% (Opcional)
Participación general. Algunos integrantes con contribuciones limitadas pero claras.	Participación desigual. Algunos integrantes con mínima contribución.	Evidencia de que 1+ integrante no participó.	Requisito de aprobación

Puntos Ponderados